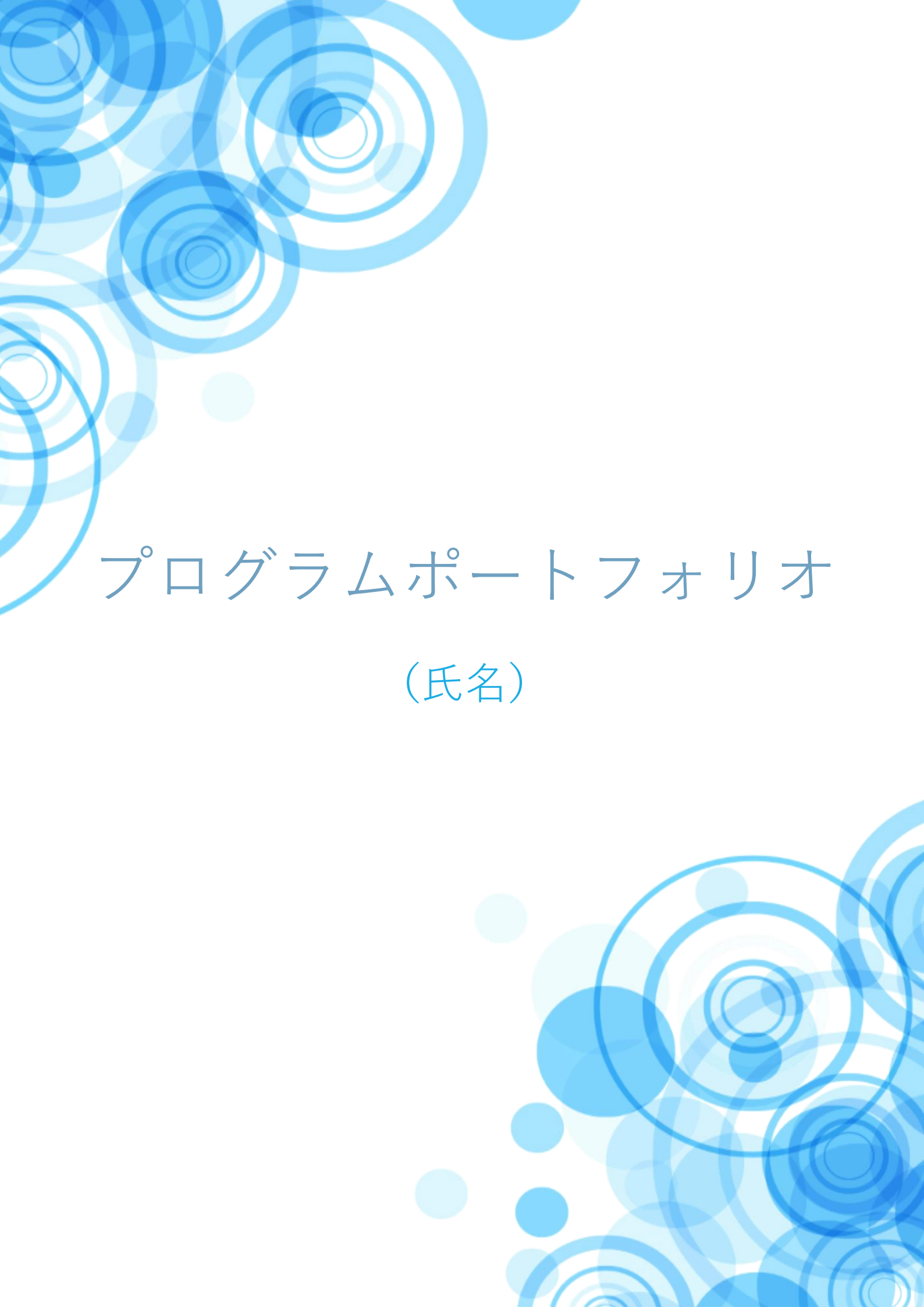




# プログラムポートフォリオ

(氏名)

# 目次

|                                               |     |
|-----------------------------------------------|-----|
| 自己紹介.....                                     | 2   |
| 学習した技術.....                                   | 2   |
| ゲームプレイ履歴.....                                 | 3   |
| Tech Stadium について.....                        | 4   |
| 作品・学習内容紹介.....                                | 5   |
| Unity ちゃんといっしょ！.....                          | 5   |
| Unity 公式チュートリアル『Tank』の改変.....                 | 1 2 |
| ファイアーレスキュー119.....                            | 1 5 |
| Alchemist（開発中）.....                           | 1 8 |
| CakePHP による住みたい街ランキング投票アプリケーション.....          | 2 0 |
| Ruby、Ruby on Rails による E-Learning システム開発..... | 2 2 |
| 終わりに.....                                     | 2 6 |

# 自己紹介

## 学習した技術

| 種類                                                              | 期間                   | 内容・作品                                                                                          |
|-----------------------------------------------------------------|----------------------|------------------------------------------------------------------------------------------------|
| Unity、C#                                                        | 6 ヶ月（2019/2～現在）      | ・ オリジナルゲームの作成<br>ポートフォリオ内に記載                                                                   |
| CakePHP、<br>phpMyAdmin                                          | 1 ヶ月（2019/7）         | 上記 Unity ゲームにおける API の作成<br>ポートフォリオに記載                                                         |
| Ruby、<br>Ruby on Rails                                          | 6 ヶ月（2018/7～2018/12） | プログラミングスクールで履修後、開発インターン参加<br>・ E-Learning システムの個人開発<br>・ 図書管理システムのチーム開発<br>個人開発についてはポートフォリオに記載 |
| Git                                                             | 1 年間（2018/9～現在）      | 上記開発業務で使用                                                                                      |
| HTML、<br>CSS、<br>JavaScript                                     | 5 年以上（2014/4～現在）     | 業務で使用。機密保持のためポートフォリオには掲載できませんが、飲食店や温泉の Web サイト、LP 制作を定期的に受注しています。                              |
| Adobe<br>InDesign、<br>Illustrator、<br>Photoshop                 | 8 年以上（2006/4～現在）     | 業務で使用。機密保持のためポートフォリオには掲載できませんが、上記 Web サイトに関わるデザインカンパ制作や、カタログ、パンフレット、取扱説明書等の DTP、画像加工を対応しています。  |
| SDL TRADOS<br>Studio、<br>Memsource、<br>Multiterm 等<br>の CAT ツール | 10 年以上（2006/4～現在）    | 業務で使用。カタログ、マニュアル、Web サイト、ソフトウェア、ゲーム UI 等のローカライズプロジェクトのマネージメント経験あり。                             |

## ゲームプレイ履歴（特に関心があったものを抜粋）

### コンシューマーゲーム

- ・『KINGDOM HEARTS』シリーズ（PS2/PS3/PS4 / SQUARE ENIX）

最も好きなゲームです。光と闇、人と人の繋がりというシンプルながらも奥深さが感じられるテーマ・世界観から、デザイン・エフェクト・音楽まで細部に渡ってこだわりが感じられる作品づくりに感動しました。

- ・『シャドウハーツII』（PS2 / アルゼ）

暗い世界観と親近感のわくキャラクター設定、史実をベースにしつつよく練られたシナリオが素晴らしいかったです。戦闘も独特で好きでした。

- ・『END OF ETERNITY』シリーズ（PS3 / トライエース）

斬新な戦闘システムが面白かったです。昼と夜で変わる街のライティングが美しく、これまで遊んだゲームの街並みで一番好きです。

### スマートフォンゲーム

- ・『ツムツム』（iOS / Disney）

シンプルですが楽しい落ちゲーでした。色々なツムを使ってみるのも楽しく、LINEの友人とスコアを競えるのも長く遊べた理由です。

- ・『Idolish7』（iOS / バンダイナムコオンライン）

楽曲が個性豊かで音ゲーとしても楽しく遊べました。シナリオも伏線が色々あり良く練られているので、現在もストーリー更新だけ追っています。

## Tech Stadium について

株式会社クリーク・アンド・リバー社運営の Tech Stadium Unity コース第 10 期を受講しました。

期間：2019 年 6 月 17 日（月）～2019 年 7 月 29 日（月）

カリキュラム：

1. クライアント技術学習（Unity）
  - ・ Git について
  - ・ Unity 基礎/基本操作
  - ・ UI（uGUI を利用した UI 作成）
  - ・ Camera 制御
  - ・ Audio 制御
  - ・ Particle System
  - ・ 接触判定（Collision、Trigger）
2. サーバー技術学習（CakePHP）
  - ・ ゲームサーバー基礎/サーバー構成
  - ・ LAMP 環境構築（PHP、MySQL）
  - ・ フレームワーク利用（CakePHP を利用した MVC、ORM）
  - ・ DB 基礎（データベース/テーブル作成、query 制御）
  - ・ Web ページ作成
  - ・ API 学習/作成
3. クライアント、サーバー連携技術学習（Unity & CakePHP）
  - ・ Unity でのデータアクセス/ネットワークアクセス
  - ・ CakePHP での API 作成、Unity との繋ぎこみ
4. その他
  - ・ マルチプレイ（Photon）
  - ・ 開発 Tips（プログラム、ツール、データベース、等）

Unity と C#による基本的なゲーム開発技術のほか、サーバ、データベースの構築と Unity 側からの GET/POST 通信についても学びました。

# 作品・学習内容紹介

Unity ちゃんといっしょ！

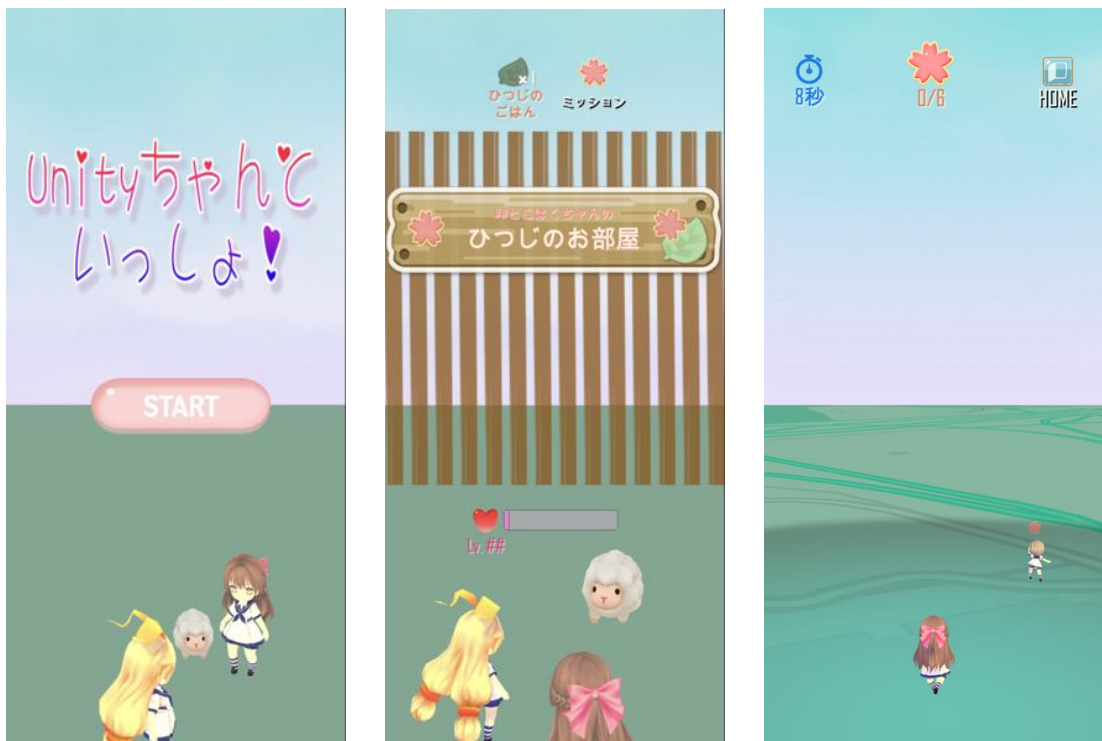
Git リポジトリ

(プロジェクト一式を git へアップロードしリンクを記載)

デモ動画

(プレイ動画を youtube へアップロードしリンクを記載)

ゲームの画像



(左上：タイトル画面。Start ボタンを押下するとユーザー登録。その後、育成画面に遷移)

(中央：育成画面。タップでひつじを撫でたり、ごはんをあげて育てることができる)

(右上：マップ画面。Google の Maps Static API に位置情報を送り、現在地に応じてマップを自動更新)



(左上：ミッション選択パネル。ミッションを選択すると、マップ画面に遷移)

(中央：ミッションパネル。パズルのピースを集めて答えを入力)

(右上：ミッションのランキングパネル。ミッションのクリアタイムが早い順番にランキングされる)

## ゲーム概要

### 1 基本説明

■ジャンル：位置情報ゲーム

■ターゲット層：小学生の女の子

■プラットフォーム：iPhone

■ 内容：

街を歩き回ってミッションをこなすことで報酬を得て、Unity ちゃんと一緒にひつじを育てる育成ゲーム。

ミッションでは、街中に散らばったヒントを入手することで、Unity ちゃんの困りごとを解決したり、暗号と一緒に解いたり、位置情報を利用した謎解き・パズルをしていきます。

## 2 ゲームの流れ

(1) 【スタートシーン】 ユーザー登録 → **育成シーン**へ遷移

(2) 【育成シーン】 できることは以下の内容

- ・ ひつじの育成（ごはんをあげる、撫でるなど）
- ・ ミッション内容確認・ランキング確認
- ・ ミッション選択 → **マップシーン**へ遷移

(3) 【マップシーン】 ミッション中の流れは以下の通り

- ・ スマホを持って街を歩き回るとヒントが出現するので、タップで取得する
- ・ ヒントを集め終わり、パズル・謎解きに正解すると、  
クリアタイムが表示され、報酬（今回はひつじのごはん）が得られる
- ・ クリアタイム登録後、ランキング確認 → **育成シーン**へ遷移

## 3 制作環境

- (1) Unity 2019.1.4f1
- (2) XAMPP ver7.3.6 / PHP ver7.3.6 / MySQL ver15.1
- (3) CakePHP 3.8、phpMyAdmin

## 制作期間

2019 年 7 月 10 日（水）～2019 年 7 月 23 日（火）の 14 日間

## 技術面

### 全体構造

シーンは Title の表示される StartScene、ひつじの育成とミッションの選択ができる BreedScene、位置情報を利用したミッションが遊べる MapScene で構成されています。ミッション選択、ステージクリア、ランキングの画面は、UI の SetActive を切り替えることで表示させています。



サーバサイドの処理はユーザー名とランキングに関する内容のみとし、育成に関連するパラメータは全て PlayerPrefs で管理し、シーン遷移前に各種パラメータの保存、シーン遷移後にロードをしています。PlayerPrefs の保存・ロード、ミッション中の時間のカウント等、全体の制御は各シーンの SceneManager で行なっています。

## 特に注意した部分

### 1. 現在地に応じてマップを取得・更新

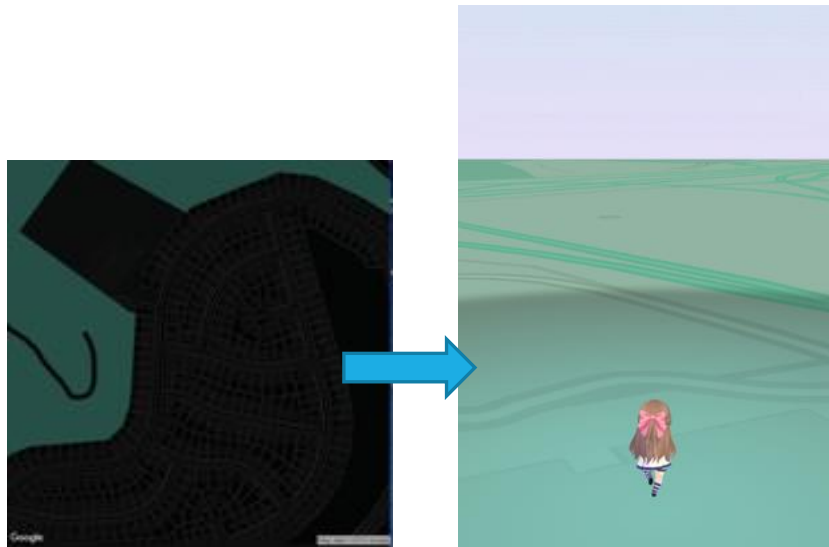
位置情報の取得・更新はオライリージャパンの『Unity による AR ゲーム開発』を参考に実装しました。

1 フレームごとに位置情報が更新されたかどうかを確認し、更新されている場合は、新しい位置情報をもとに画像をリクエスト、Map\_Tiles オブジェクトの地図上の古いタイルテクスチャを破棄し、取得した画像を新しいテクスチャとして設定しています。

```
55 void Update ()
56 {
57     //新しい位置が取得されたかチェック
58     if (gpsLocationService != null &&
59         gpsLocationService.IsServiceStarted &&
60         lastGPSUpdate < gpsLocationService.Timestamp)
61     {
62         lastGPSUpdate = gpsLocationService.Timestamp;
63         worldCenterLocation.Latitude = gpsLocationService.Latitude;
64         worldCenterLocation.Longitude = gpsLocationService.Longitude;
65         print("GoogleMapTile refreshing map texture");
66         RefreshMapTile();
67     }
68 }
69
70 public void RefreshMapTile() {
71     StartCoroutine(_RefreshMapTile());
72 }
73
74 IEnumerator _RefreshMapTile ()
75 {
76     //タイルの中心の緯度・経度を取得
77     tileCenterLocation.Latitude = GoogleMapUtils.adjustLatByPixels(worldCenterLocation.Latitude, (int)(size * 1 * TileOffset);
78     tileCenterLocation.Longitude = GoogleMapUtils.adjustLonByPixels(worldCenterLocation.Longitude, (int)(size * 1 * TileOffset);
79
80     var queryString = "";
81
82     //マップタイルをリクエストするqueryStringパラメータを作成
83     queryString += "center=" + WWW.UnEscapeURL (string.Format ("{0},{1}", tileCenterLocation.Latitude, tileCenterLocation.Longitude));
84     queryString += "&zoom=" + zoomLevel.ToString ();
85     queryString += "&size=" + WWW.UnEscapeURL (string.Format ("{0}x{0}", size));
86     queryString += "&scale=" + (doubleResolution ? "2" : "1");
87     queryString += "&maptype=" + mapType.ToString ().ToLower ();
88     queryString += "&format=" + "png";
89 }
```

(GoogleMapTile：マップ更新処理の一部)

また、マップのテクスチャについて、本のまま実装すると黒地に蛍光緑の SF 調の雰囲気のマップになってしまうので、Google Maps Platform 公式の Developer Guide を読み、 明度を反転させる Invert\_lightness を false にし、また Unity 側でも Map\_Tile のマテリアルにトゥーンシェーダを使用して、のどかな雰囲気のマップになるよう調整を入れました。



(左図：スタイル、シェーダ調整前) (右図：スタイル、シェーダ調整後)

## 2. アニメーション・エフェクト

プレイヤーがボタンをタップしたり、画面遷移した時に起こるアニメーションやエフェクトの細部にこだわって実装しています。ミッション中のパズルのパネルは Scale 0 から 1 に拡大しつつ現れるようにアニメーションを設定し、またプレイヤーがひつじに対して撫でる（タップ）や「ごはんをあげる」際の一連のアニメーション、エフェクトは、必要に応じて Coroutine を使用して自然な流れになるよう調整を入れました。

```

28
29
30 public void OnClickTakeFood()
31 {
32     BreedSceneManager.FoodAmount--;
33     BreedSceneManager.FoodAmountText.text = string.Format("{0}", BreedSceneManager.FoodAmount);
34
35     BreedSceneManager.food.SetActive(true);
36     BreedSceneManager.sheep.SetTrigger("stun");
37     BreedSceneManager.heart.SetActive(true);
38     BreedSceneManager.BreedSlider.value += 30;
39
40     StartCoroutine("Vanished");
41 }
42
43 IEnumerator Vanished()
44 {
45     // 1.5秒間処理を止める
46     yield return new WaitForSeconds(1.5f);
47
48     // 0.5秒後食事を消す
49     BreedSceneManager.food.SetActive(false);
50     BreedSceneManager.heart.SetActive(false);
51     yield return new WaitForSeconds(0.5f);
52     BreedSceneManager.FoodConfirmation.SetActive(false);
53 }

```

(ひつじに食事を与えた後の挙動の制御)

### 3. PlayerPrefs とサーバの GET/POST 通信

ランキング機能の実装とひつじのステータス保存のために PlayerPrefs とデータベースと活用しています。

PlayerPrefs に関しては、各シーンの遷移前にユーザー名、ひつじのステータス (Lv、スケール)、アイテム個数を保存し、シーン遷移後にロードする処理を SceneManager に入れています。

サーバ側では、ID、ユーザー名、クリアタイムを管理し、データベース更新時のキーとなる「ユーザー名」は重複しないようバリデーションを設定し、重複している場合は、クライアント側で再入力を促すメッセージが表示されるようにしました。また、ランキングについては、クリアタイムが NULL のレコードは除外されるようにサーバ側で処理を入れています。

```
76 .....//Puzzleのクリアタイムランキング取得
77 .....public function getPuzzlesystem()
78 .....{
79 .....    error_log("getPuzzlesystem()");
80 .....    $this->autoRender = false;
81 .....
82 .....    //-----
83 .....    //クエリを利用してクリアタイムがNullでないレコードを取得する
84 .....    $query = $this->Usertable->find()
85 .....    ->where(function($exp){return $exp->isNotNull('Cleartime');});
86 .....
87 .....    //データの並べ替えを行う
88 .....    $query->order(['Cleartime' => 'ASC']); .....// 'Id' を昇順にソート
89 .....    $query->limit(10);
90 .....
91 .....    //クエリを元にjson形式へ変換
92 .....    $json_array = json_encode($query);
93 .....
94 .....    // 内容出力
95 .....    echo $json_array;
96 .....}
```

(クリアタイムが NULL のレコードを除外する処理)

### 4. 世界観の統一

課題制作ではありましたが、自作ゲームのような雰囲気ではなく、APP ストアに並ぶような UI の再現、世界観の統一を目指したいという気持ちがありました。そのため、ゲームのターゲット層、世界観を明確にし、そこから逆算する形で UI、配色、使用するフォント、モデルにこだわって制作しています。

ランキングには ScrollView を使用し、整った可愛いレイアウトになるように実装しました。



```

54 private void CallbackWwwSuccess(string response)
55 {
56     // json データ取得が成功したのでデシリアライズして整形し画面に表示する
57     List<RankingData> rankingList = RankingDataModel.DeserializeFromJson(response);
58
59     int i = 1;
60     foreach (RankingData rank in rankingList)
61     {
62         var item = GameObject.Instantiate(prefab) as RectTransform;
63         item.SetParent(Content.transform, false);
64
65         var No = item.GetChild(3).gameObject.GetComponent<Text>();
66         No.text = i.ToString();
67
68         var Name = item.GetChild(1).gameObject.GetComponent<Text>();
69         Name.text = $"{rank.Name} ";
70
71         var Cleartime = item.GetChild(2).gameObject.GetComponent<Text>();
72         Cleartime.text = $"{rank.Cleartime}";
73         i++;
74     }
75 }
76

```

(データベースから取得したレコードを整形して表示する処理)

## Unity 公式チュートリアル『Tank』の改変

### ゲームの画像



(左図：開始時に地雷が一瞬表示される様子)



(右図：被弾時にシールドが展開する様子)

### ゲーム概要

Unity 公式チュートリアルに従って Tank プロジェクトを作成し、地雷とシールド機能を実装しました。地面に埋め込まれた地雷に接触すると一定期間動けなくなる機能と、所定のキーを押すと一定期間シールドが発動し、攻撃を弾くという機能です。シールド展開時は砲弾の発射ができなくなる仕様にし、シールドが自弾によって被弾しないよう調整を行いました。

### 制作期間

2019 年 6 月の 2 日間ほど

### 技術面

インプットに Guard キーの設定を追加し、キーが入力された際にシールドが表示されるようにしました。また、シールド自体に Collider を設定し、Shell に衝突した場合のみエフェクトや衝突音等の効果が表示されるようにしました。地雷についても同様に Collider を設定し、Player が衝突した場合に Active になり、Player の動きが一定期間止まるよう Coroutine で処理を入れています。

またゲーム開始時に一時的に地雷の位置が見えるよう、GameManager にも処理を追加しました。

```

7      public int m_PlayerNumber = 1;
8      public GameObject Sheild;
9      private string m_GuardButton;
10
11      void Start()
12      {
13          m_GuardButton = "Guard" + m_PlayerNumber;
14      }
15
16      void Update()
17      {
18          StartCoroutine(Guard());
19      }
20
21      IEnumerator Guard()
22      {
23          if (Input.GetButton(m_GuardButton))
24          {
25              Sheild.SetActive(true);
26              yield return new WaitForSeconds(3.0f);
27              Sheild.SetActive(false);
28          }
29      }

```

(TankGuard：ガードボタン押下時の処理)

```

4      public class ShockController : MonoBehaviour
5      {
6          public GameObject m_Shock;
7
8          void OnTriggerEnter(Collider other)
9          {
10             if (other.gameObject.tag == "Player")
11             {
12                 StartCoroutine(Shock(other.gameObject));
13             }
14         }
15
16         IEnumerator Shock(GameObject hitTank)
17         {
18             m_Shock.SetActive(true);
19             hitTank.GetComponent<TankMovement>().m_Speed = 0.0f;
20
21             yield return new WaitForSeconds(2.0f);
22             m_Shock.SetActive(false);
23             hitTank.GetComponent<TankMovement>().m_Speed = 12.0f;
24         }

```

(ShockController：地雷接触時のスクリプト)

```

1  using UnityEngine;
2
3  public class GuardController : MonoBehaviour
4  {
5      public GameObject m_ExplosionPrefab;
6
7      private AudioSource m_ExplosionAudio;
8      private ParticleSystem m_ExplosionParticles;
9
10     private void Awake()
11     {
12         m_ExplosionParticles = Instantiate(m_ExplosionPrefab).GetComponent<ParticleSystem>();
13         m_ExplosionAudio = m_ExplosionParticles.GetComponent<AudioSource>();
14
15         m_ExplosionParticles.gameObject.SetActive(false);
16     }
17
18     void OnTriggerEnter(Collider other)
19     {
20         if (other.gameObject.tag == "Shell")
21         {
22             m_ExplosionParticles.transform.position = transform.position;
23             m_ExplosionParticles.gameObject.SetActive(true);
24
25             m_ExplosionParticles.Play();
26
27             m_ExplosionAudio.Play();
28             Destroy(other.gameObject);
29         }
30     }

```

(GuardController：シールドが Shell に接触した際の処理)

## ファイアーレスキュー119

### Git リポジトリ

有料アセットを使用しているため非公開

### ゲームの画像



### ゲーム概要

チームで開発した VR ゲームです。パートナー2名はゲームステージ、NPC 制作担当で、私はゲームクライアント、UI、プレイヤー制作と全体の進行管理を担当しました。

#### 1 基本説明

■ジャンル：シューティング

■ターゲット：小学生・中学生（学校で消防訓練を受ける年頃の子供）

■プラットフォーム：Oculus Quest



## ■コンテンツ内容・目的：

「火災現場で覚える消火器の使い方」というコンセプトで開発しました。

従来の消防訓練のVRは教育用ということもあり固い雰囲気のものが多く、子供に自発的に消火器の使い方に慣れてもらい、火災現場の雰囲気に吞まれないように教育する方向性ではないという点に課題を感じていました。このVRでは、シューティングゲームの要素を取り込むことで、子供がゲームを楽しみながら、消火器の使い方を覚えられるコンテンツとすることを目的に開発しました。

## 2 ゲームの流れ

### (1) 【スタートシーン】学校のグラウンドでメニュー表示

- ・チュートリアル（消火器の使い方とゲーム内の操作説明）
- ・ゲームスタート

### (2) 【ゲームシーン】消火器を使い、校内の火災を避けつつ逃げ遅れたクラスメートを救出

- ・ゲーム終了条件：
  - ① 所定の救助人数を救助する（ゲームクリア）
  - ② 体力が0になる（ゲームオーバー）
  - ③ 所定の救助人数を達成する前に制限時間が0になる

### (3) 【リザルト表示】残り秒数＋獲得スコアで総合評価表示

## 3 制作環境

Unity 2018.2.21f1、Oculus Quest

## 制作期間

2019年6月1日（土）～2019年6月15日（土）の14日間

## 技術面

シーンは Title の表示される StartScene、ゲームが遊べる GameScene で構成されています。リザルト画面は、GameScene のまま UI の SetActive を切り替えることで表示させています。スコア、時間管理など、全体の監督は GameController で行われ、リザルト画面は獲得スコアにより 5 段階評価で表示されます。

```
33     private int GradeS = 200;
34     private int GradeA = 180;
35     private int GradeB = 150;
36     private int GradeC = 120;
37     private int GradeD = 100;
38     private bool gameover = false;
39
40     void Start()
41     {
42         player = GameObject.FindGameObjectWithTag("Player").GetComponent<Player>();
43         this.initParameter();
44
45         Score = GameObject.Find("Score").GetComponent<Text>();
46         Score.text = "SCORE:" + score;
47     }
48
49     void Update()
50     {
51         //消火器に追従するUIの管理
52         lifeGage.fillAmount = player.life / player.maxLife;
53         leastTime = Maxtime - Mathf.CeilToInt(Time.timeSinceLevelLoad);
54         timeGage.fillAmount = (float)leastTime / (float)Maxtime;
55         Countdown.text = leastTime + "秒";
56         HelpedPeople.text = "救助人数" + player.on_touch + "/" + player.NPC + "人";
57
58         //救助人数を満たした際のリザルト画面表示
59         if (leastTime == 0 || player.NPC == player.on_touch && Fin == false)
60         {
61             Fin = true;
62             GUI.SetActive(false);
63             LaserPointer.SetActive(false);
64             EventSystem.SetActive(true);
65             OVRGazePointer.SetActive(true);
66             Result.SetActive(true);
67             FinalScore.text = "スコア : " + score;
68             LeftTime.text = "残り時間:" + leastTime;
69             EvaluationPoints.text = (score + leastTime).ToString();
70
71             if (score + leastTime >= GradeS && !gameover)
72             {
73                 Gameclear_comment.text = "You clear!!";
74                 Fires.SetActive(false);
75                 Grade.text = "S";
76                 Grade_comment.text = "avior!!";
77                 Comments.text = "すばらしい手際のよさです。\\n実際の火災現場でも冷静な判断を心がけましょう";
78             }
79             else if (score + leastTime >= GradeA && score + leastTime <= GradeS && !gameover)
80             {
81                 Gameclear_comment.text = "You clear!!";
82                 Fires.SetActive(false);
83                 Grade.text = "A";
84                 Grade_comment.text = "wesome!";
85                 Comments.text = "素晴らしい手際のよさです。\\n実際の火災現場でも冷静な判断を心がけましょう";
86             }
87             else if (score + leastTime >= GradeB && score + leastTime <= GradeA && !gameover)
88             {
89                 Gameclear_comment.text = "You clear!!";
90                 Fires.SetActive(false);
91                 Grade.text = "B";
92                 Grade_comment.text = "verage!";
93                 Comments.text = "まあまあいい手際のよさです。\\n実際の火災現場でも冷静な判断を心がけましょう";
94             }
95             else if (score + leastTime >= GradeC && score + leastTime <= GradeB && !gameover)
96             {
97                 Gameclear_comment.text = "You clear!!";
98                 Fires.SetActive(false);
99                 Grade.text = "C";
100                Grade_comment.text = "verage!";
101                Comments.text = "まあまあいい手際のよさです。\\n実際の火災現場でも冷静な判断を心がけましょう";
102            }
103            else if (score + leastTime >= GradeD && score + leastTime <= GradeC && !gameover)
104            {
105                Gameclear_comment.text = "You clear!!";
106                Fires.SetActive(false);
107                Grade.text = "D";
108                Grade_comment.text = "verage!";
109                Comments.text = "まあまあいい手際のよさです。\\n実際の火災現場でも冷静な判断を心がけましょう";
110            }
111            else
112            {
113                Gameclear_comment.text = "You clear!!";
114                Fires.SetActive(false);
115                Grade.text = "F";
116                Grade_comment.text = "verage!";
117                Comments.text = "まあまあいい手際のよさです。\\n実際の火災現場でも冷静な判断を心がけましょう";
118            }
119        }
120    }
```

(GameController：ゲーム中とりザルト画面の UI 更新部分)

教室や廊下に発生する火には Collider を設定してあり EnemyController で自動生成されています。消火器を持つ左コントローラから Raycast が出て、火にフォーカスを当てた状態でトリガーを引くと、消火器から水が噴射するエフェクトとともに火が消え、スコアが加算されます。

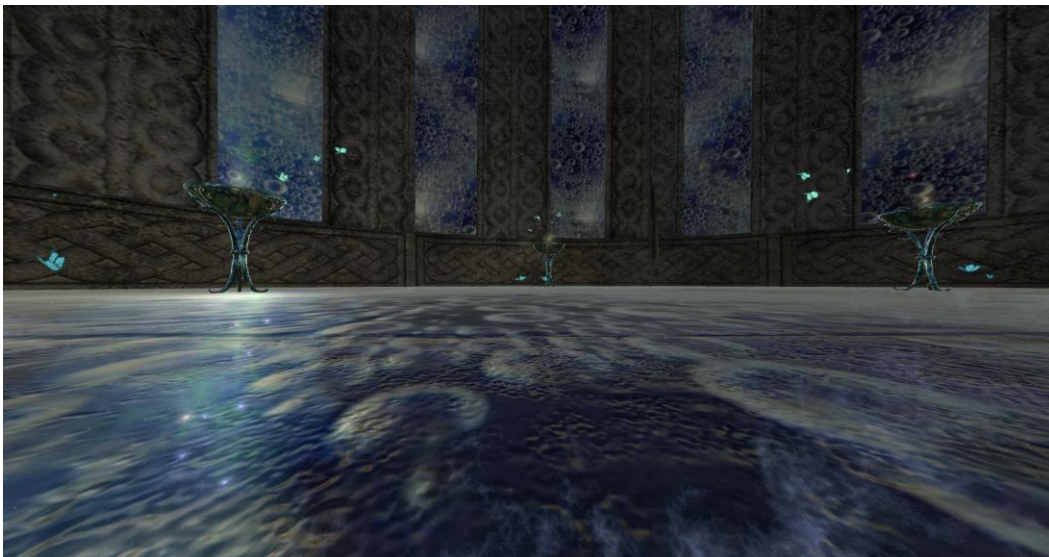
残り時間、救助人数などの UI は常にプレイヤーの視界に入るよう、消火器に追従する形で表示させており、体力や残り時間は直感的に把握できるようにスライダーで可視化しています。

## Alchemist（開発中）

Git リポジトリ

有料アセットを使用しているため非公開

ゲームの画像



（塔の内部：開発中の画面です）

## ゲーム概要

8 月末に完成予定で現在、個人で開発中の VR ゲームです。

### 1 基本説明

■ジャンル：アドベンチャー

■ターゲット：VR を初めて体験する人

■プラットフォーム：Oculus Quest

■コンテンツ内容・目的：

宇宙と繋がっているような不思議な塔を探索しつつ性格診断をするコンテンツです。

VR の一人称視点のゲームにおける、体験者が「見知らぬ空間」に「前提知識なく投げ出された」という特異なシチュエーションと不安感をシナリオに利用しています。体験者は精神的な迷子として塔の中の探索し、不思議な塔の住人との対話を通して、「本当の自分」を探っていきます。各部屋の選択によって性格診断のパラメータをプラスしていき、最上階にたどり着くと性格診断の結果を受け取ることができます。

### 2 制作環境

Unity 2018.2.21f1、Oculus Quest

## 制作期間

2019 年 7 月～2019 年 8 月 24 日（土）の約 1 ヶ月半

## 技術面

このゲームは世界観にこだわって制作をしているため、2019 年 7 月現在はステージを重点的に作り込んでいく段階です。Skybox には銀河のテクスチャを使用し、塔の壁と床のマテリアルは銀河が透けて見えるよう Rendering Mode を Transparent に設定しました。この設定により描画が非常に重くなったため、不要なオブジェクトを削除、マテリアルやメッシュをまとめる Unity のアセット Mesh Baker を利用してポリゴン数

を削減、プレイヤーの視界に入らない階層は SetActive を false にする等、工夫して FPS を改善しています。

## CakePHP による住みたい街ランキング投票アプリケーション

### 画像

Sumimachi

関東住みたい街ランキング2019

投票ページ

| 街名   | 代表的な沿線名 |                                   |
|------|---------|-----------------------------------|
| 横浜   | JR東海道本線 | <input type="button" value="投票"/> |
| 恵比寿  | JR山手線   | <input type="button" value="投票"/> |
| 吉祥寺  | JR中央線   | <input type="button" value="投票"/> |
| 大宮   | JR京浜東北線 | <input type="button" value="投票"/> |
| 新宿   | JR山手線   | <input type="button" value="投票"/> |
| 目黒   | JR山手線   | <input type="button" value="投票"/> |
| 武蔵小杉 | 東急東横線   | <input type="button" value="投票"/> |
| 鎌倉   | 江ノ島電鉄線  | <input type="button" value="投票"/> |
| 品川   | JR山手線   | <input type="button" value="投票"/> |
| 浦和   | JR京浜東北線 | <input type="button" value="投票"/> |

ホーム

結果を見る

Sumimachi

関東住みたい街ランキング2019

人気No.1は横浜、1222票です！（2019年07月25日現在）

| 順位  | 街名  | 代表的な沿線名 | 票数   |
|-----|-----|---------|------|
| 1 位 | 横浜  | JR東海道本線 | 1222 |
| 2 位 | 恵比寿 | JR山手線   | 877  |
| 3 位 | 吉祥寺 | JR中央線   | 774  |
| 4 位 | 大宮  | JR京浜東北線 | 567  |
| 5 位 | 新宿  | JR山手線   | 552  |

ホーム

### 概要

Tech Stadium Unity コースにおける Web アプリケーション課題で制作。

「街名」「沿線名」「票数」が登録されたデータベースに対し、投票で票数を更新し、遷移後の画面で 5 位までランキング表示します。

### 制作環境

- (1) XAMPP ver7.3.6 / PHP ver7.3.6 / MySQL ver15.1

## (2) CakePHP 3.8、phpMyAdmin

### 技術面で特に注意した点

result（結果表示）と update（データベース更新）の切り分け

投票しなくても結果だけ見ることができるように、Controllerで update と result を分け、結果を見る場合は result に遷移、投票後に結果画面を表示する場合は update 後に result にリダイレクトするようにしました。

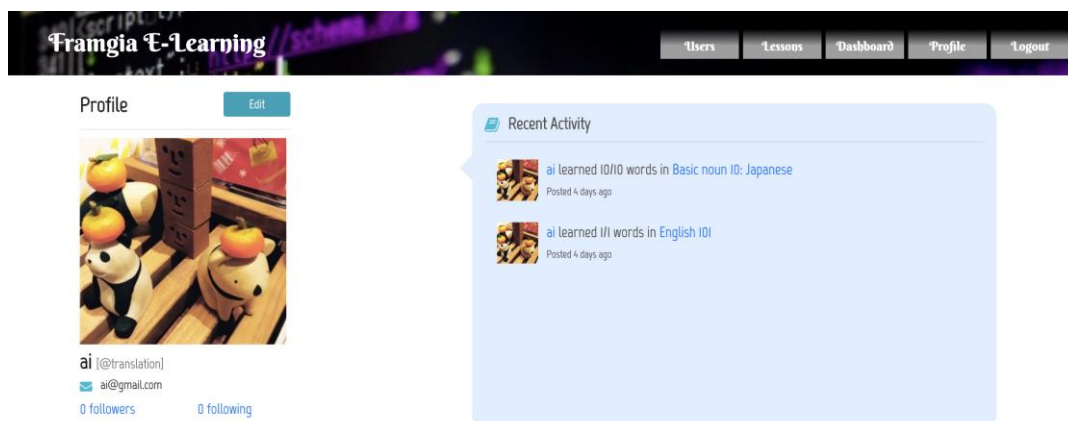
```
21     ...public function result()
22     ...{
23         ...//Sumimachi テーブルの取得と並び替え
24         ...$SumimachiTable = $this->Sumimachi->find("all");
25         ...$SumimachiTable->order(['Popularity' => 'DESC']);
26         ...$SumimachiTable->limit(5);
27         ...$arrayA = $SumimachiTable->toArray();
28
29         ...$this->set('arrayA', $arrayA);
30     ...}
31
32     ...public function update()
33     ...{
34         ...//Post データの取得
35         ...$postData = $this->request->getData('id');
36
37         ...//レコードの更新
38         ...$SumimachiTable = $this->getTableLocator()->get('sumimachi'); //Sumimachi テーブルの取得
39         ...$targetRecord = $SumimachiTable->get($postData); //Sumimachi テーブルから該当IDのレコードを取得
40         ...$targetRecord->Popularity = $targetRecord['Popularity'] + 1; //レコードのPopularityを更新
41         ...$SumimachiTable->save($targetRecord); //更新したレコードの保存
42
43         ...//result にリダイレクト
44         ...return $this->redirect(['controller' => 'Sumimachi', 'action' => 'result']);
45     ...}
```

# Ruby、Ruby on Rails による E-Learning システム開発

## Git リポジトリ

(プロジェクト一式を git へアップロードしリンクを記載)

## 画像



(プロフィール画面)



(左：単語テスト画面、右：スコア表示画面)

## 概要

プログラミングスクールの最終課題の個人開発で、Ruby、Ruby on Rails、部分的に JavaScript を使用して、E-Learning システムの開発をしました。主な機能として、ユーザー登録、ユーザー同士のフォロー、プロフィール登録、テスト、テスト結果の確認、Administrator によるテスト作成およびロール管理があります。

## 制作環境

Ruby ver 2.5.1 / Ruby on Rails ver 5.2.1 / MySQL

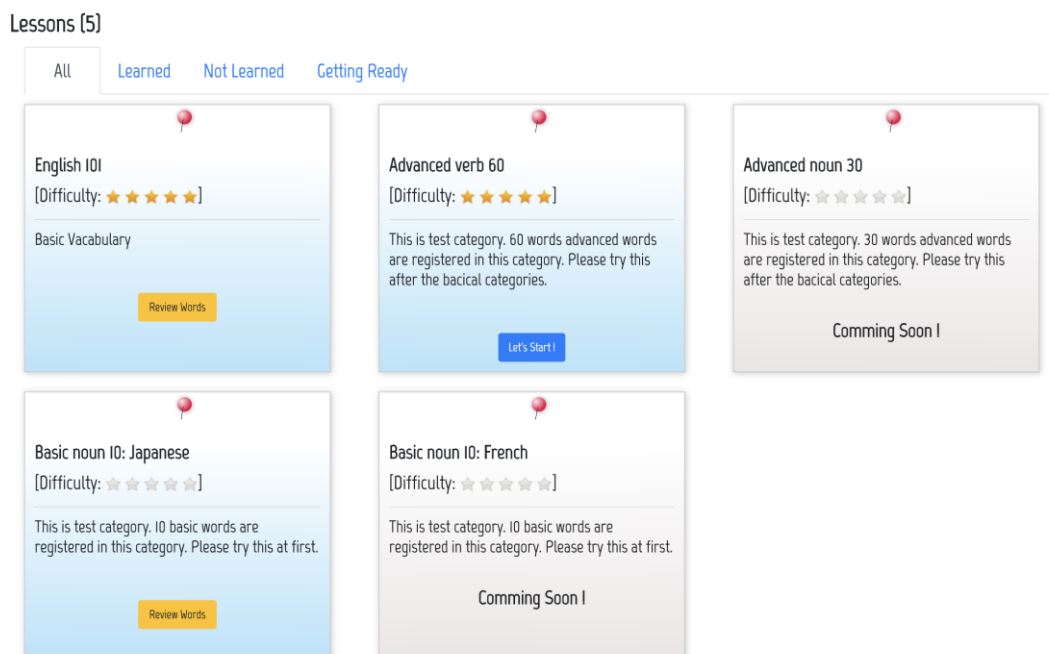
## 開発期間

2018 年 9 月の約 1 ヶ月間

## 技術面で特に注意した点

### 1. レッスン選択画面の表示切り替え

レッスン選択画面では、(1) 各ログインユーザーがすでに受験済みのテスト、(2) 未受験のテスト、(3) Administrator 側でカテゴリ登録のみ済んでいて、テストの作成が完了していないテストが可視化されるよう、View で条件を分けて表示されるようにしました。



(レッスン選択画面)



```

<h3 class="mb-5"><%= category.description %></h3>

<div class="text-center">
  <% if category.words.empty? %>
    <h2><strong>Comming Soon !</strong></h2>
  <% elsif @lessons.pluck(:category_id).any?(category.id) %>
    <% @lesson_test = @lessons.find_by(category_id: category.id) %>
    <%= link_to "Review Words", category_lesson_answers_path(category_id: category.id, lesson_id: @lesson_test.id) %>
  <% else %>
    <%= link_to "Let's Start !", category_lessons_path(category_id: category.id), method: :post, class: "btn btn-primary" %>
  <% end %>
</div>

```

(categories/index.html.erb 内の条件分岐)

## 2. ロール管理機能 (Super-Admin、Admin、User 権限) の設定

この E-Learning システムでは、テストを受ける「User」、テストを作成する「Administrator」 (Admin のメンバーリスト画面で「User」にロール変更可能)、「Administrator」と同様の権限を持ち「User」へのロール変更ができない上位管理者「Super-Administrator」の3段階のロール管理をする必要がありました。

このため、User テーブルに「authority」カラムを追加し、Admin::UsersController、CategoriesController クラス等の before\_action でログインユーザーのロール確認を入れ、「Administrator」以上の権限を持っていない場合は User 用のページにリダイレクトする処理を入れています。

```

1  class Admin::UsersController < ApplicationController
2    before_action :require_login
3    before_action :administrator
4
5    def index
6      @users = User.paginate(page: params[:page], per_page: 10)
7    end
8
9    def update
10     @user = User.find(params[:id])
11     @user.authority = User::ROLE_ADMIN
12     @user.save
13     redirect_to admin_users_path
14   end
15
16   def destroy
17     @user = User.find(params[:id])
18     @user.authority = nil
19     @user.save
20     redirect_to admin_users_path
21   end
22
23   private
24
25   def administrator
26     unless current_user.authority?
27       redirect_to users_path
28     end
29   end
30 end

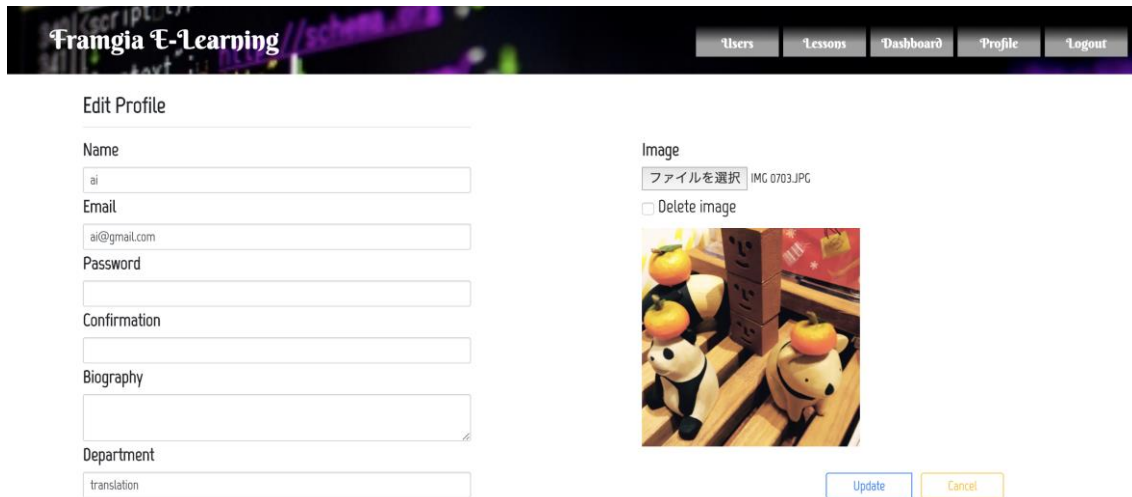
```

(admin/users\_controller.rb 内のロール確認)

### 3. 使用感、品質の向上のために JavaScript 使用

サーバ課題ではありますが、使用感を向上させるため部分的に JavaScript を使用しています。

**プロフィール画面：** Rails のみで実装すると編集画面上でのプロフィール画像のプレビュー表示ができないため、JavaScript でプレビュー機能を実装しました。



(プロフィール編集画面)

```
44 <div class="col-md-5 center-block">
45   <%= f.label :image %><br>
46   <%= f.file_field :image, id: :user_image %>
47   <p><%= f.check_box :remove_image %> Delete image</p>
48   <% if @user.image? %>
49     <%= image_tag @user.image.url, id: :img_prev %>
50   <% else %>
51     <%= image_tag @user.image.url, id: :img_prev%>
52   <% end %>
53
54   <script type="text/javascript">
55     $(function() {
56       function readURL(input) {
57         if (input.files && input.files[0]) {
58           var reader = new FileReader();
59           reader.onload = function (e) {
60             $('#img_prev').attr('src', e.target.result);
61           }
62           reader.readAsDataURL(input.files[0]);
63         }
64       }
65       $("#user_image").change(function(){
66         readURL(this);
67       });
68     });
69   </script>
```

(Users/edit.html.erb の JavaScript 部分)

**レッスン画面のテスト難易度**：星の数で難易度を表現する機能は実装方法を調べ、jQuery の Raty を使用して実装しました。星の数は User 側のレッスン選択画面では ReadOnly（編集不可）とし、Administrator 画面でのみ編集できるようにしています。

```
<% else %>
  <% @categories.each do |category| %>

    <div class="col-md-4 mb-3 mt-3">
      <div class="<%= category.words.any? ? "lesson-box" : "lesson-box-prep" %>">
        <h2><strong><%= category.title %></strong></h2>
        <p>[Difficulty:
        <span class='star-rating' data-score="<%= category.difficulty %>"></span>]</p><hr>

        <script>
          $('<%= category.words.any? ? "lesson-box" : "lesson-box-prep" %>').raty({
            readOnly: true,
            path: '/assets/',
            click: function(score, e) {
              $("#difficulty_rating").val(score)
            }
          });
        </script>
      <h3 class="mb-5"><%= category.description %></h3>
    </div>
  </div>
```

(categories/index.html.erb の一部)

# 終わりに

幼い頃からゲームが好きで、ゲーム開発に携わりたいと思っていました。

ゲームの専門学校に行くことができず一度は諦めたのですが、2018年にフリーランスになり、セブでRubyの勉強をしつつ、休学して学びに来ている大学生や社会人、異なる価値観をもつ現地の人々、異文化のなかで様々な年代、経歴の方々と触れ合うなかで、私からは自然と作りたいゲームのアイデアを話すことが増え、日本に戻ってゲーム開発を学ぼうと思うようになりました。

Unityで拙いながらもゲームを作り出すと、これまで自分が遊んできた数々のゲームの細部のこだわり、再現したくても今の自分にはどうしたら作れるのかわからない技術に目が向くようになりました。Tech Stadium Unityコースで学び、講師の方々のサポートでそのいくつかを自分の手で実装し、新しい技術を身につけながらひとつのゲームを完成させたことに強い喜びを感じています。

回り道をしています、これまで仕事を通して得たコンテンツ編集、ローカライズ、プロジェクト管理のスキルはゲーム業界でも活かせるのではないかと考えています。ゲームエンジニアとして、これから身に付けるべきことが山ほどあると思いますが、ゲーム業界で経験を積み、これまで夢中になった数々のゲームに比肩する作品を生み出す一助となれるよう、技術を磨いていきたいと思っています。

最後まで読んでいただきありがとうございました。

以上